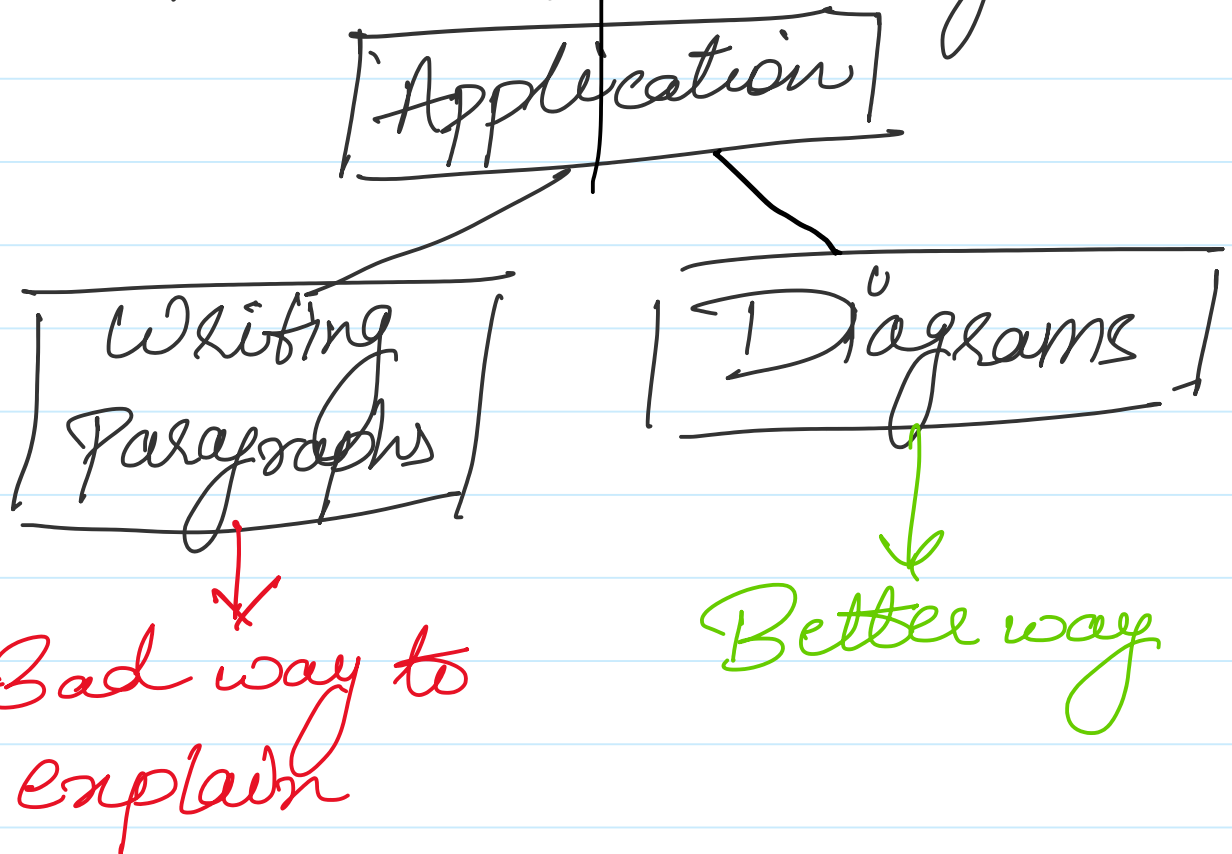


Introduction

Wednesday, 16 September 2026

9:35 PM

Suppose you are making an application. How will you explain your application to a friend. 2 ways:-

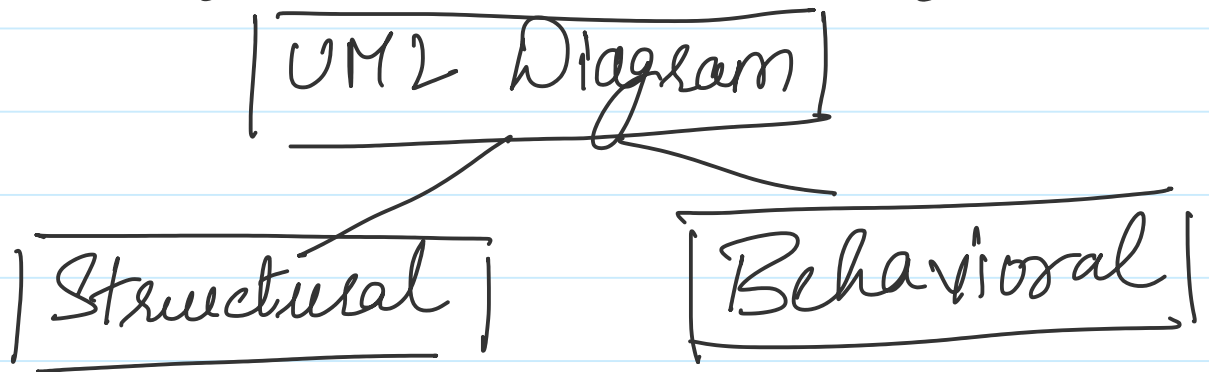


UML Diagrams :-

- You have rough idea of your Application.
- You express your idea in diagrams on paper / computer.
- In that diagram :-
 - Component
 - Objects / Entities
 - How the objects are related

- Objects / Entities
- How objects are related to each other.

Two types of UML Diagrams :-



Structural :-

Defines structure of our application

- what components are there
- how they are connected

also called static Diagram.

Behavioural :-

Defines how different component ^{or} object are interacting.

- how different object are sending message to each other.
- how one object is calling method of other object.

also called Dynamic Diagram.

There are 7 Structural and 7 Behavioural UML Diagram are there.

But we will study only (2).

But we will study only (2).

Other 12 not used practically.

↳ Used in other use -
cases - NOT IMPORTANT types

2 UML Diagrams that we have to study

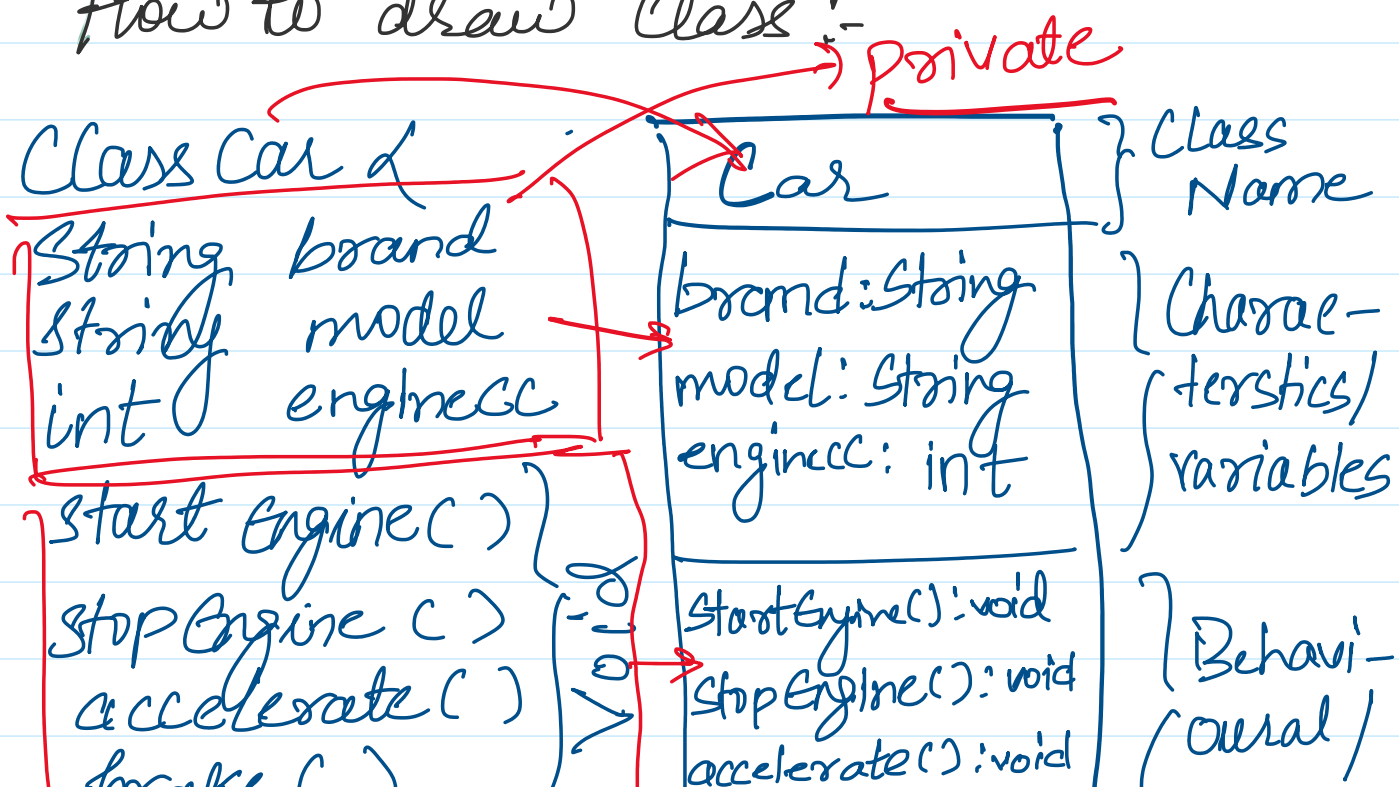
1. Class Diagram - Structural
2. Sequence Diagram - Behavioural

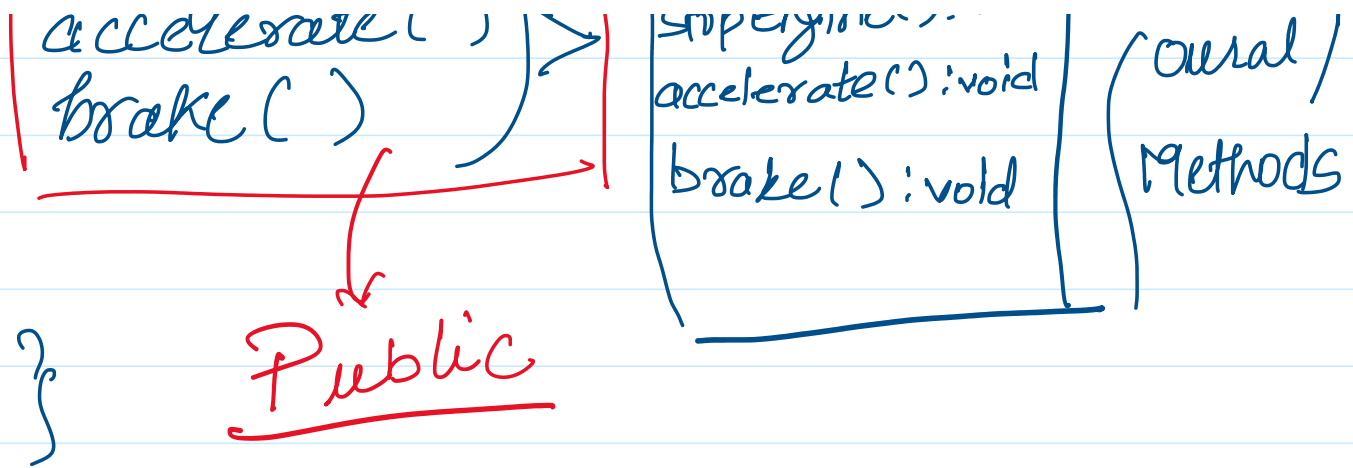
CLASS diagram:-

Represents all classes in our application and how they are connected / associated.

- ① Class structure
- ② Association / connections.

How to draw Class:-



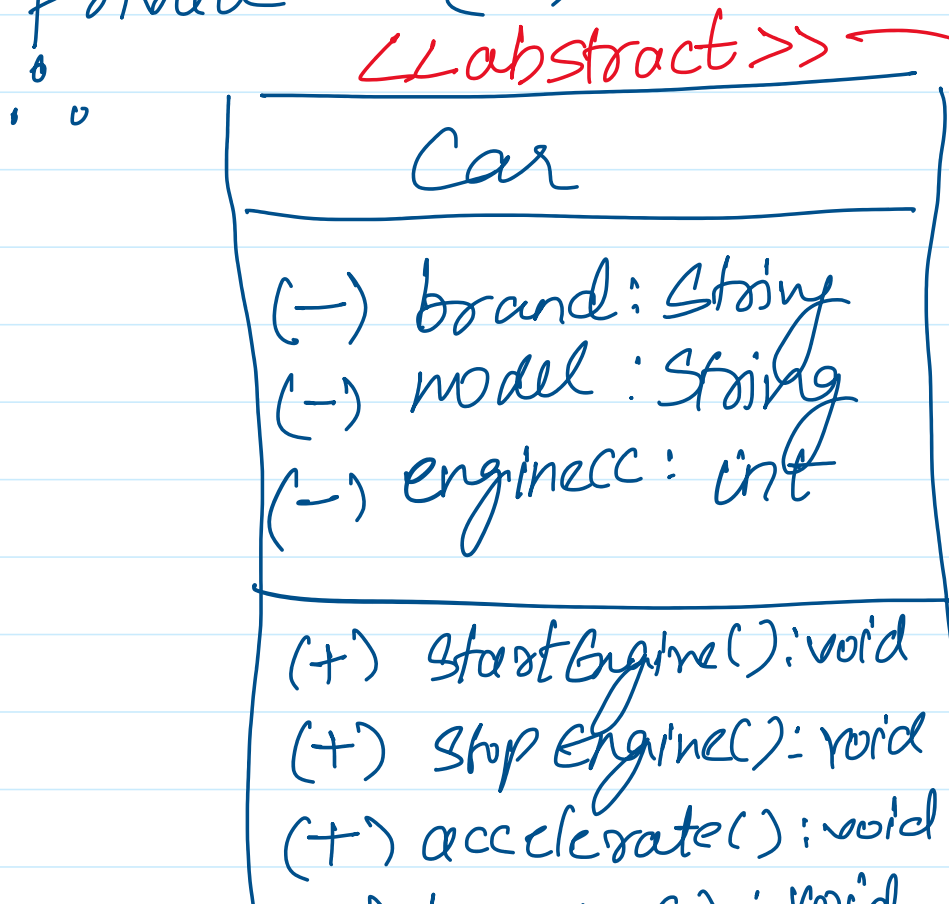


Access Modifiers

Wednesday, 16 September 2026 10:08 PM

Access Modifiers	within class	from child class	outside the class
public	(✓)	(✓)	(✓)
Protected	(✓)	(✓)	(X)
Private	(✓)	(X)	(X)

public — (+) protected — (#)
private — (-)



<<abstract>>

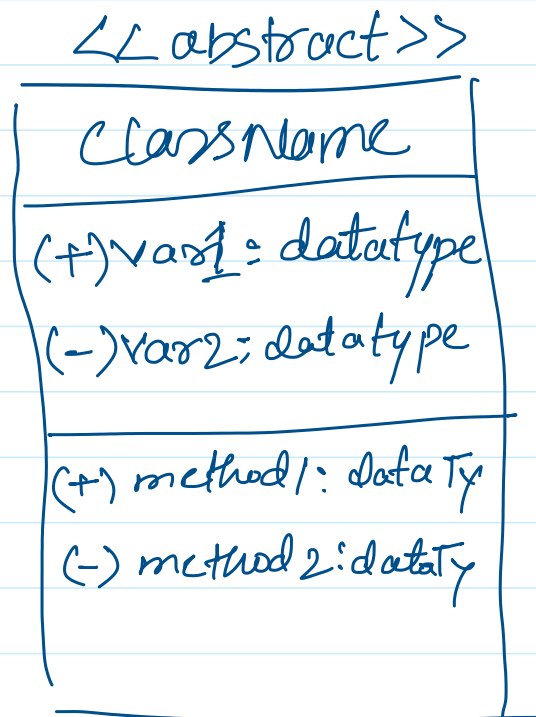
if this is a abstract class.

for normal class we don't write anything.

(+) accelerate(): void
(+) brake(): void

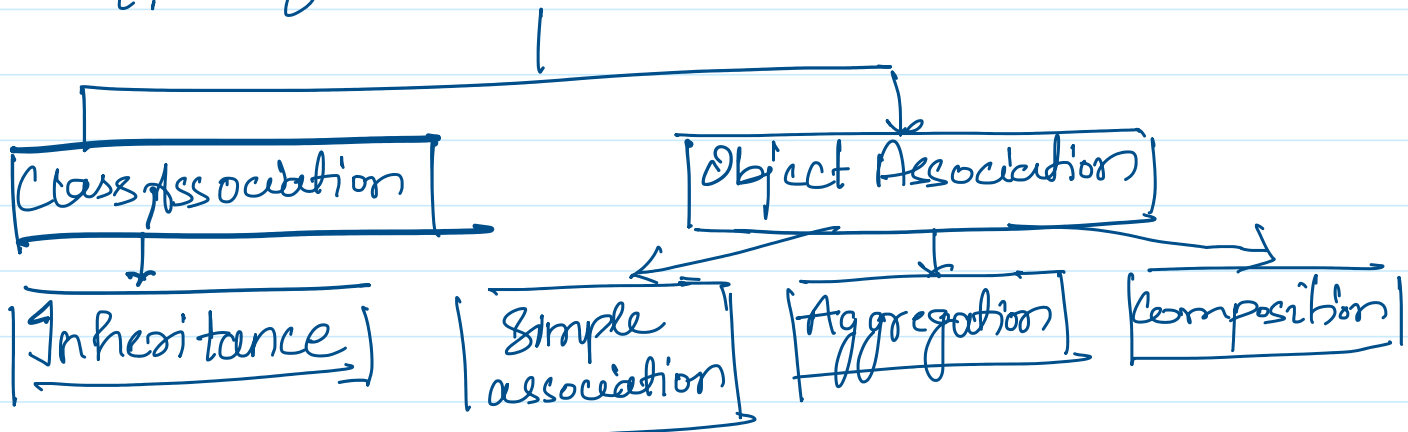
anything.

Class diagram looks like:-



Association:-

Types of Association



Combining called
COMPOSITION

Why they all are called composition?
 b/c their implementation ^{/programmatically} is same but theoretically they are different.

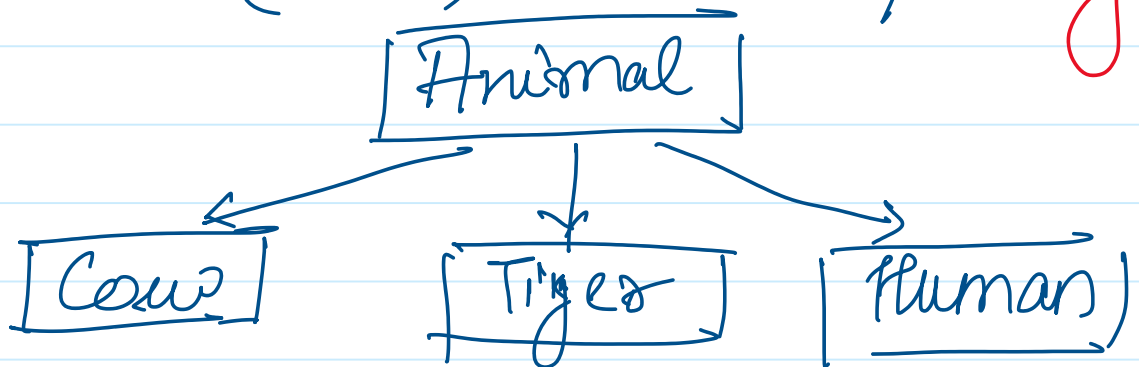
Inheritance

```
Class A {
    method1();
}
```

```
Class B : Public A {
    method2();
}
```

```
main() {
    B * b = new B();
    b -> method1();
    b -> method2();
}
```

It is a (is-a) relationship. *why?*

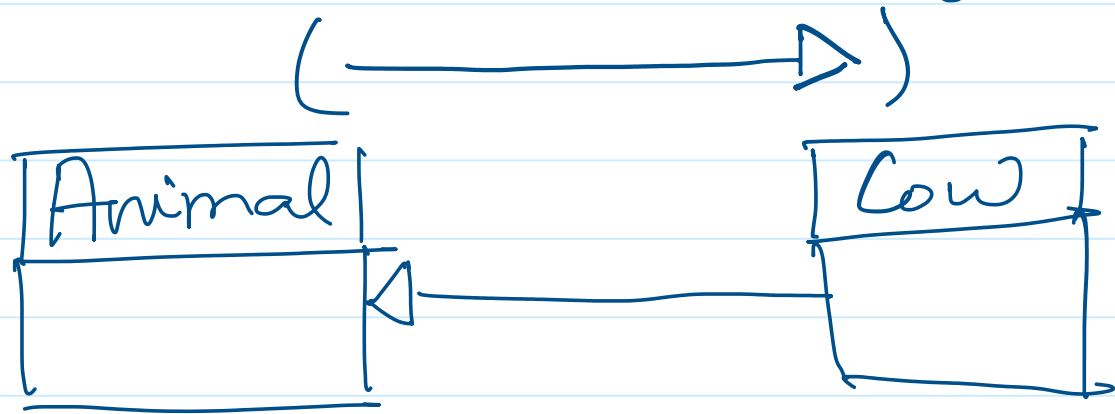


Cow is-a Animal

In UML Diagram:-

(is-a) is represented by :-

(1:5-a) is represented by :-



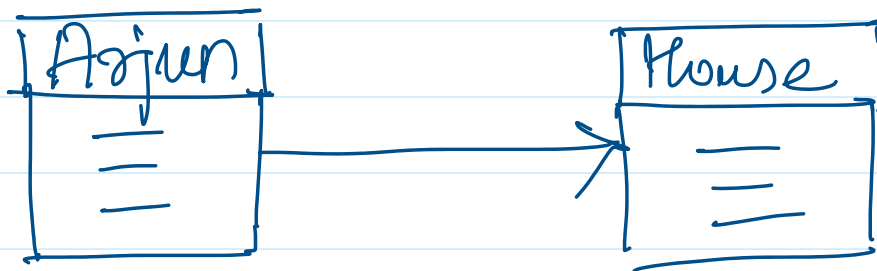
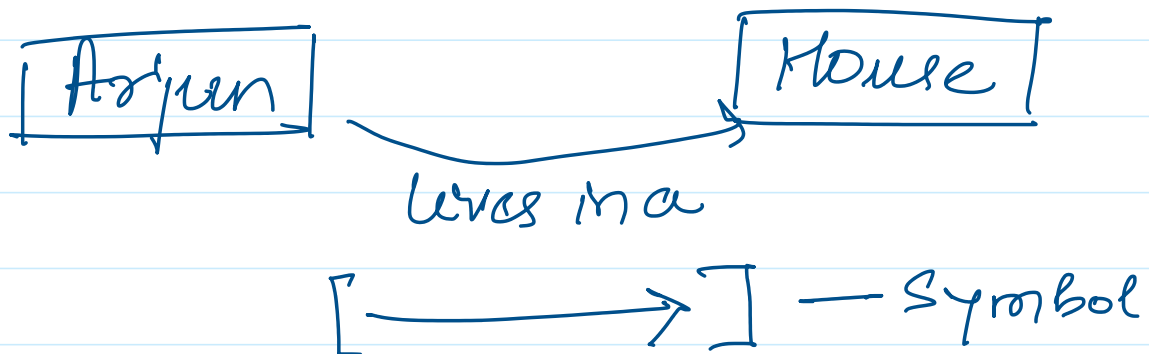
Composition

↳ (has-a) relationship

All (3) types of composition has (has-a) relationship

In program all 3 are represented same
Just there is a difference of ~~ATC~~

— Simple Association.

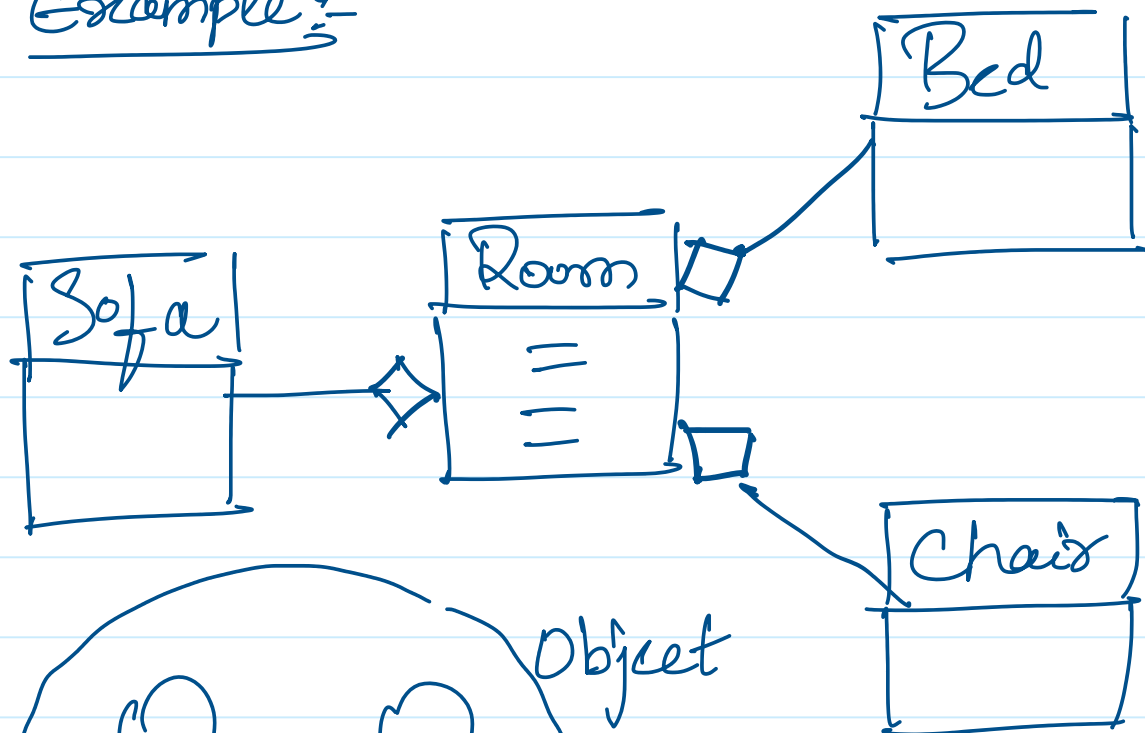


Simple
Association

— Aggregation (has-a) relationship

There is a parent object in which multiple object lives.

Example:-



This is much **stronger** than Simple Association

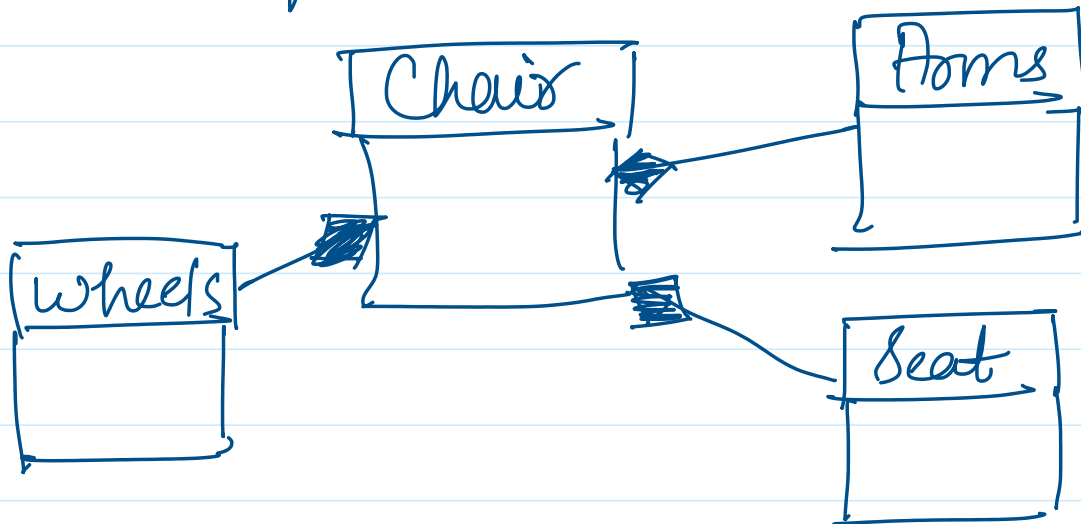
[—◇] — Symbol

In this a chair class can exist individually

for this a class can exist individually.

— Composition (has-a relationship)

It is the strongest form of relation b/w object.



Here chair is made up of different parts. These parts can't exist individually.

[—●] — Symbol

CODE :-

```

class A {
    method1();
}
  
```

```

class B {
    A * a
  }
  
```

```

B() { a = new A(); }
  
```

```

main() {
    B * b = new B();
    b->method2();
    b->a->method1();
}
  
```

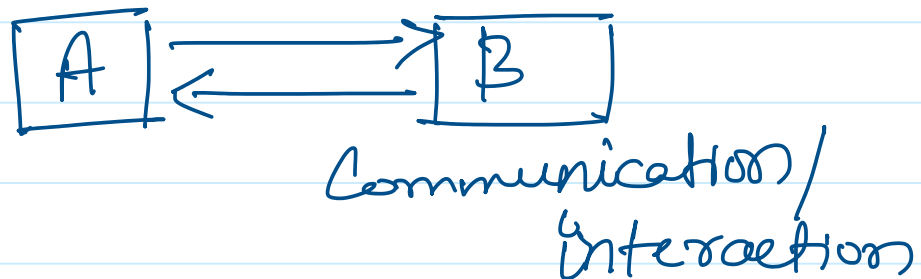
```
1 1 1 1  
B() & a=new A();  
method2();  
}
```

Note:- Sometimes you will be confused to pick the relationship b/w Aggregation & Composition
— Pick anyone you feel right

↓
No answer is wrong or right, it is just your viewpoint

SEQUENCE Diagram

- How 2 classes interact with each other
- when the interaction will happen.



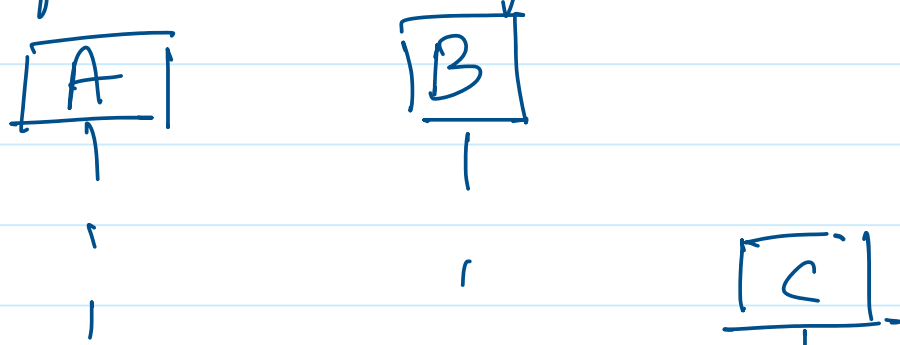
It is simply represented as

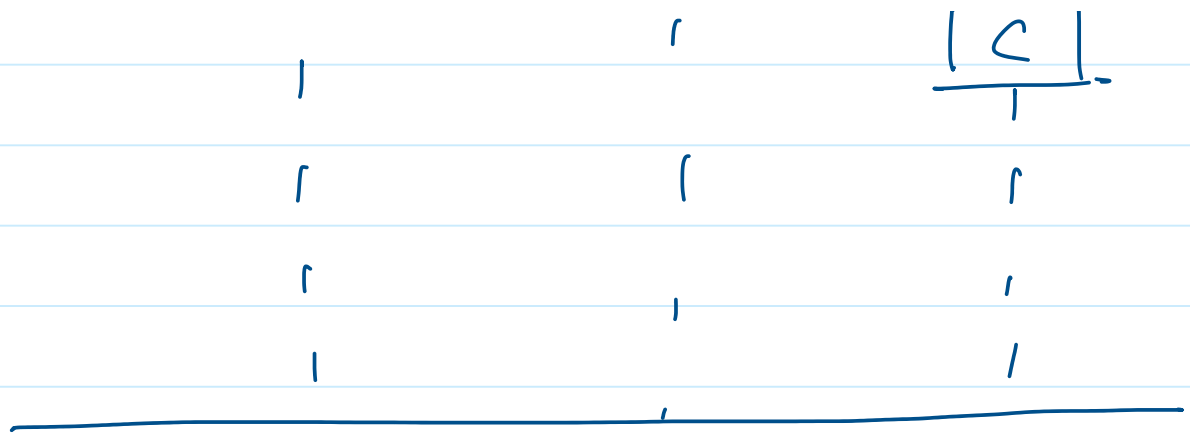


Terms

1. Lifeline - Tell us at what time a particular object, entity or component will exist in our application.

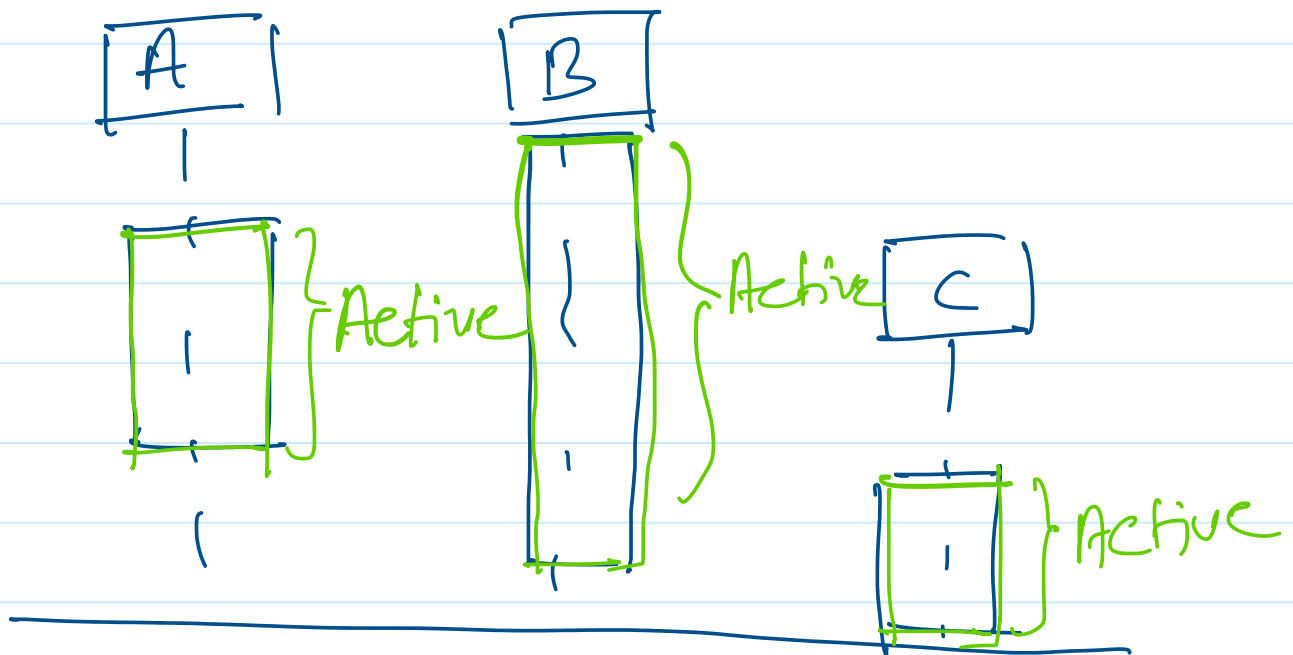
Example - two object are there





2- Activation Bar -

Till when the object will be active.



In active mode object can send and receive message.

3. Messages

Two types of message :-

Async

Here you keep on sending message

Sync

Here you wait for the response to send next

sending
message
back to back

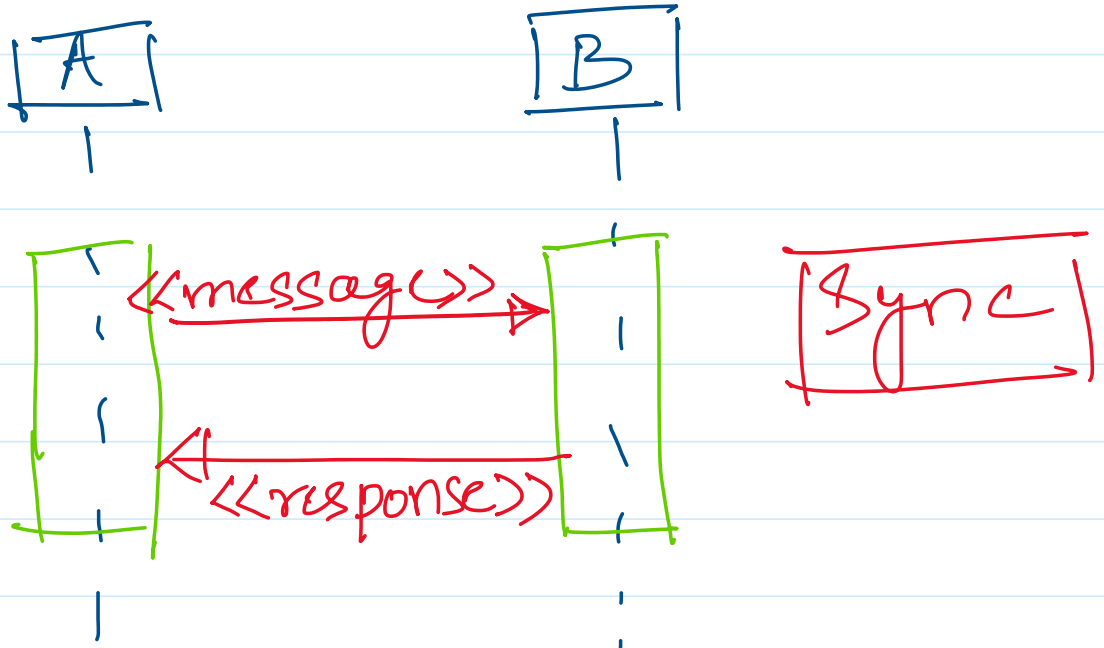
to send next
message.

Don't wait

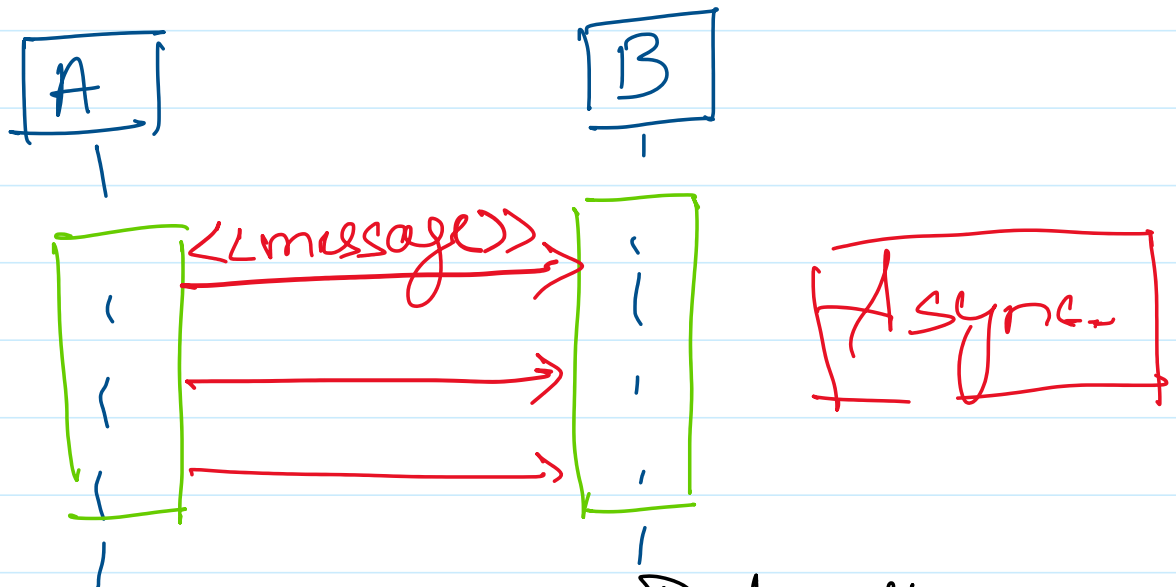
Sync and Asynchronous

Wednesday, 16 September 2026 11:26 PM

Sync :-



Asyne :-



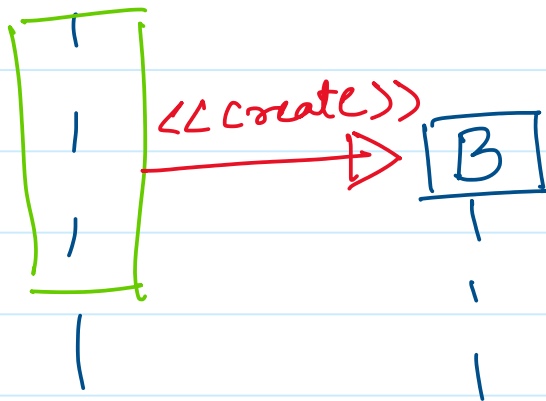
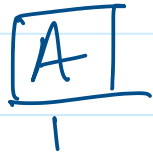
Create Message

Destroy Message

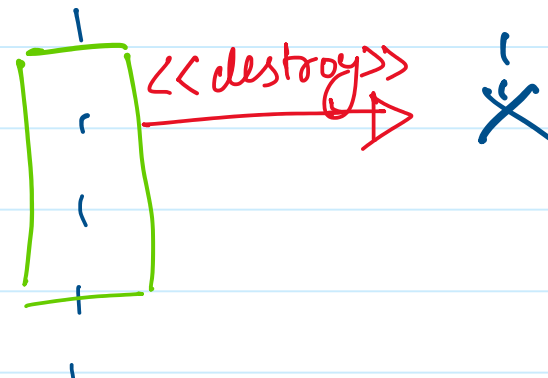
When application is running then there will be situation when a object/Entity/Component is created or destroyed.

Component is created or destroyed.
To create or destroy a message is
sent called create message or
destroy message.

Create Message v/s



Destroy message

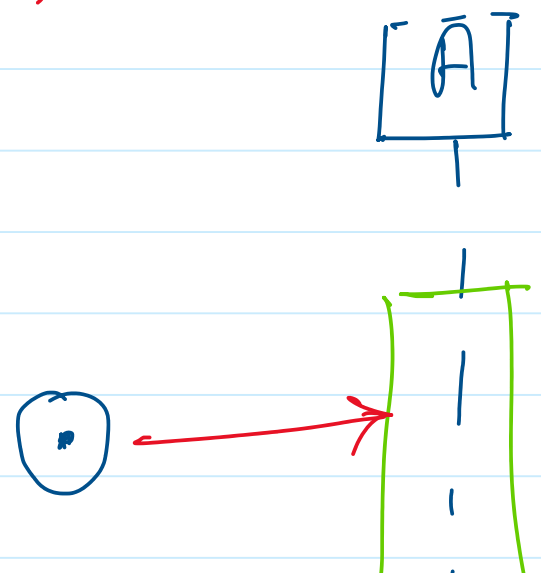
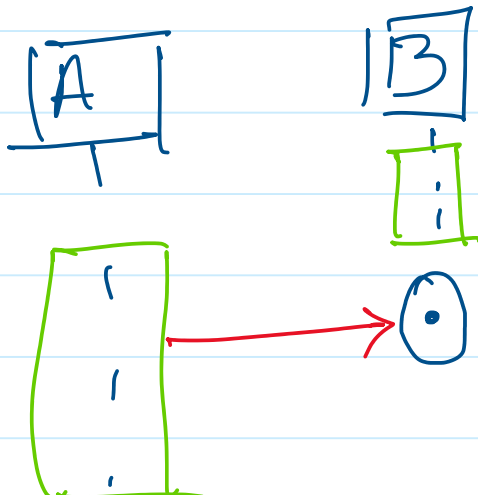


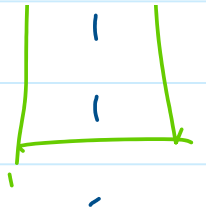
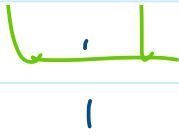
Lost Message v/s

Found Message

Message which is not received by anyone is called lost message.

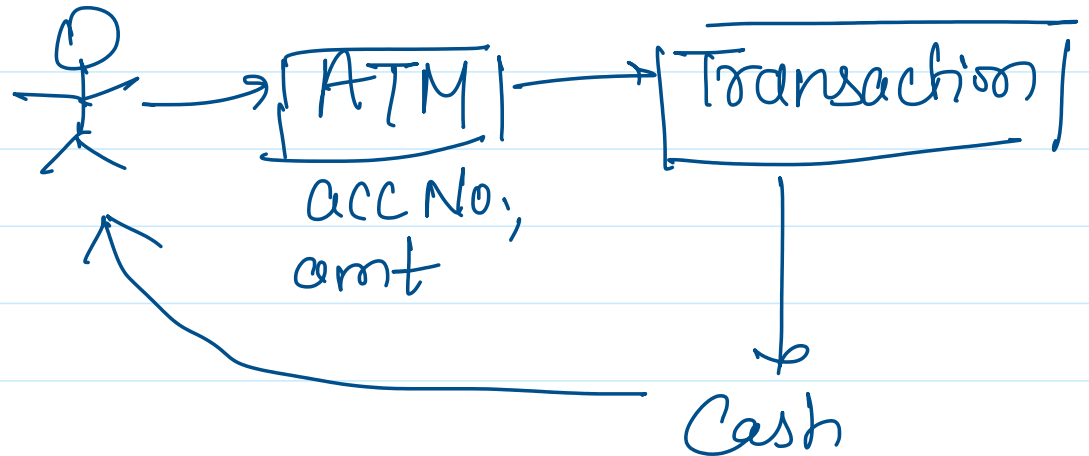
Message whose source is un-known (don't know from where it is coming) is called found Message.



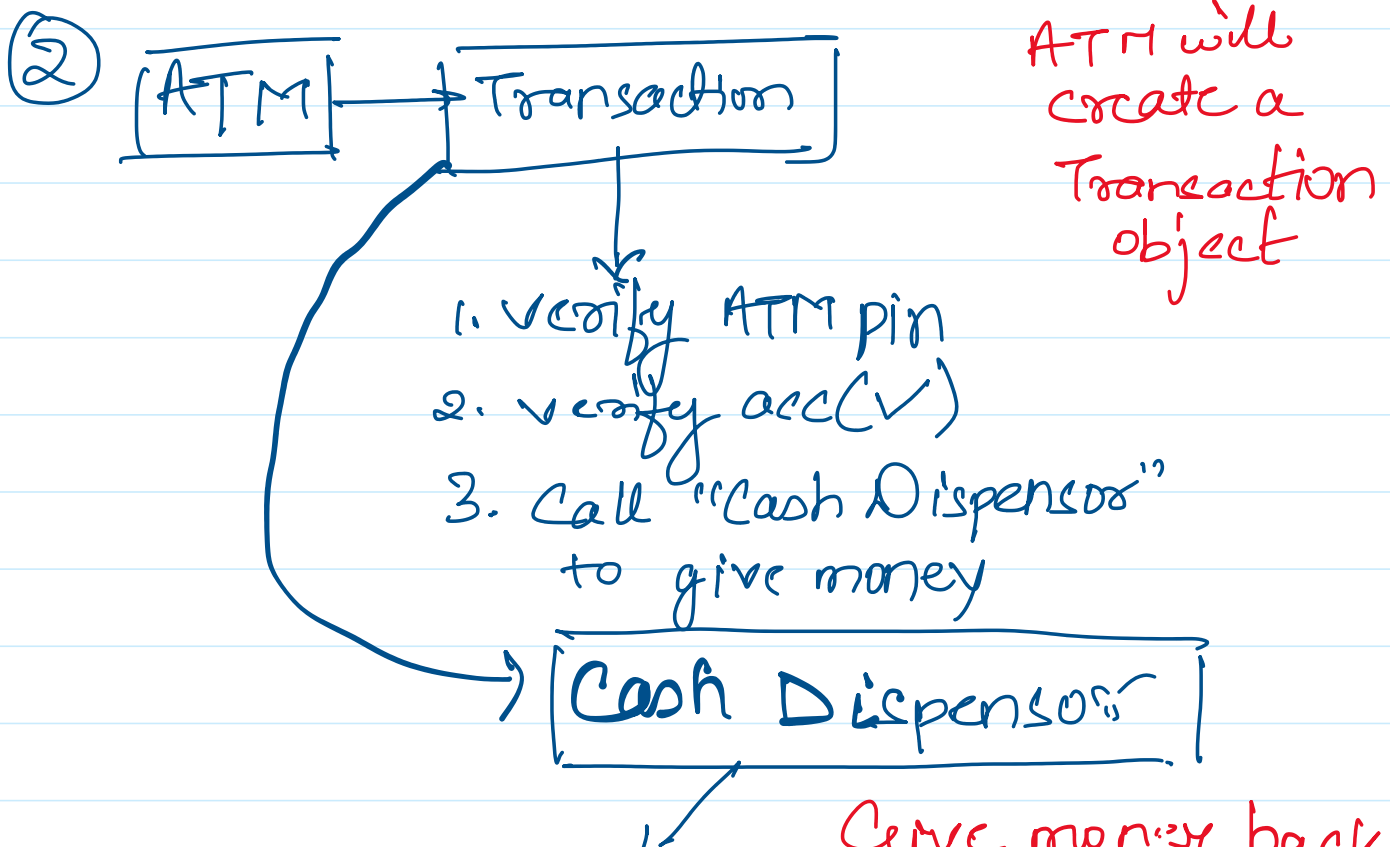
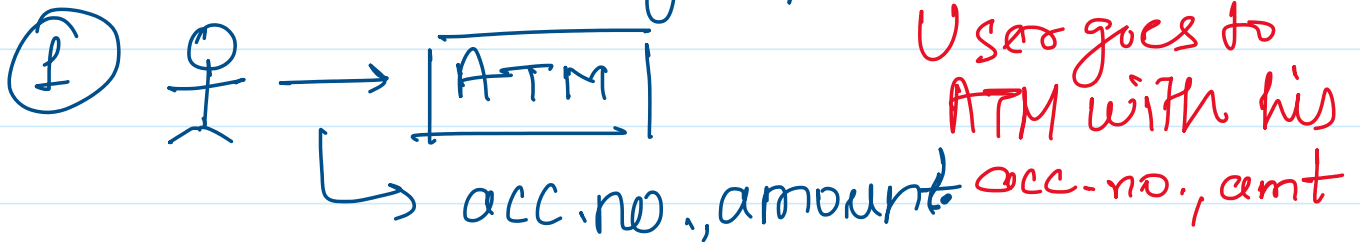


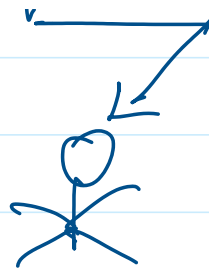
How to Draw?

Example



Use Case - Step By step.

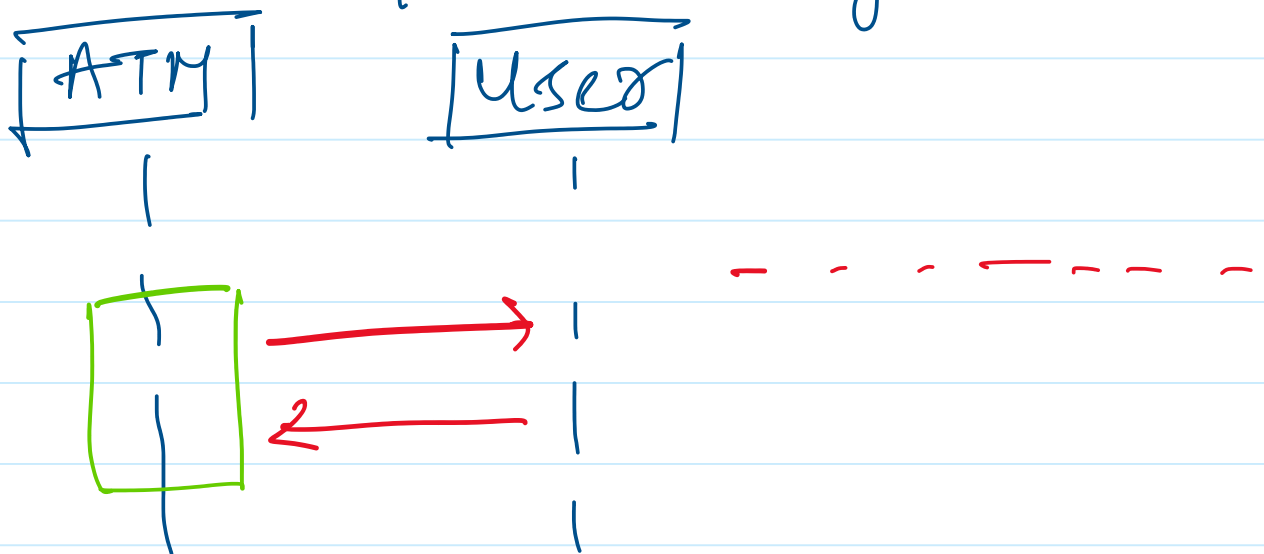




Give money back
to user.

Sequence diagram :-

1. Use case — already studied
2. Identify objects → ATM
 - User
 - Transaction
 - Account
 - Cash Dispensor
3. Draw Sequence Diagram.

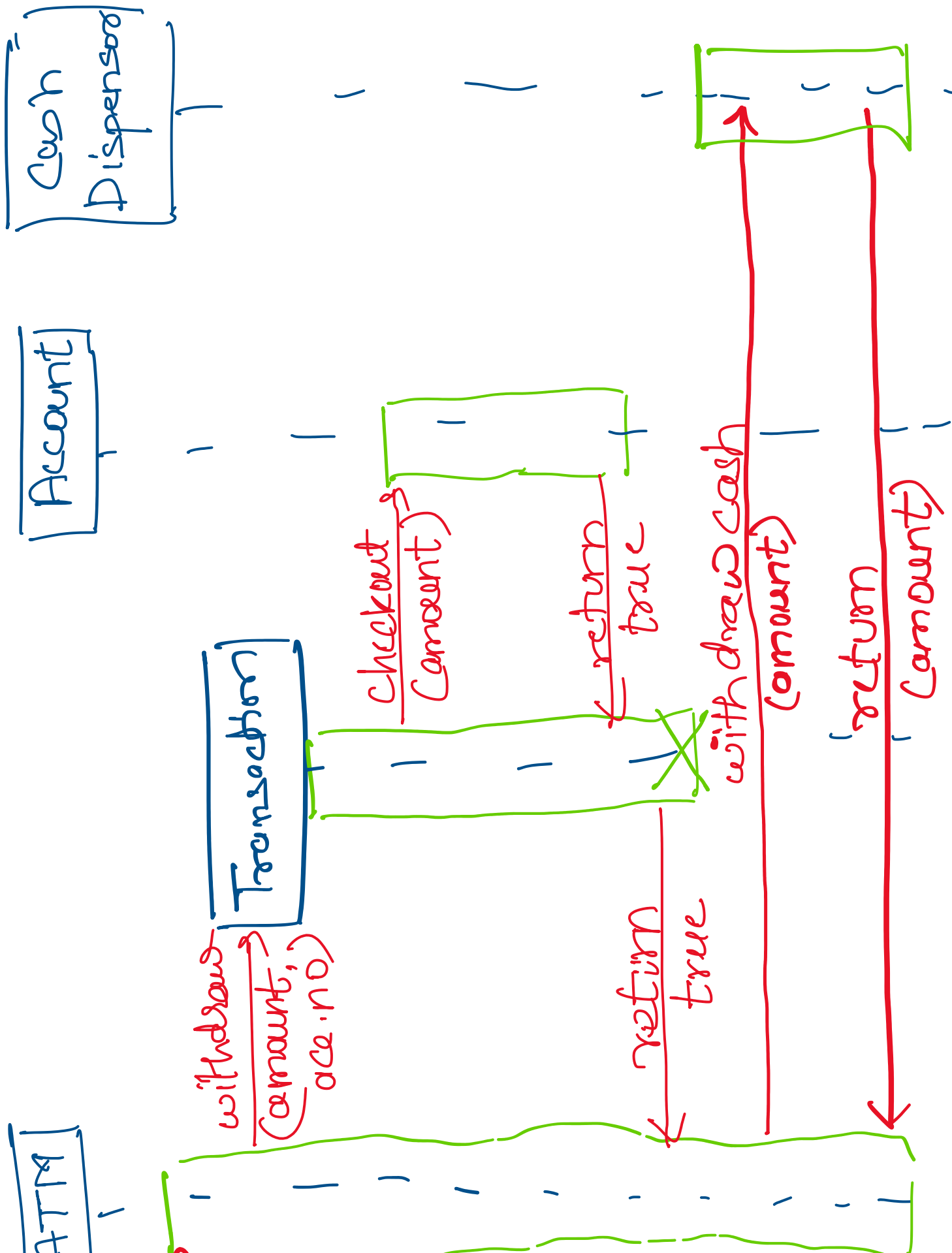


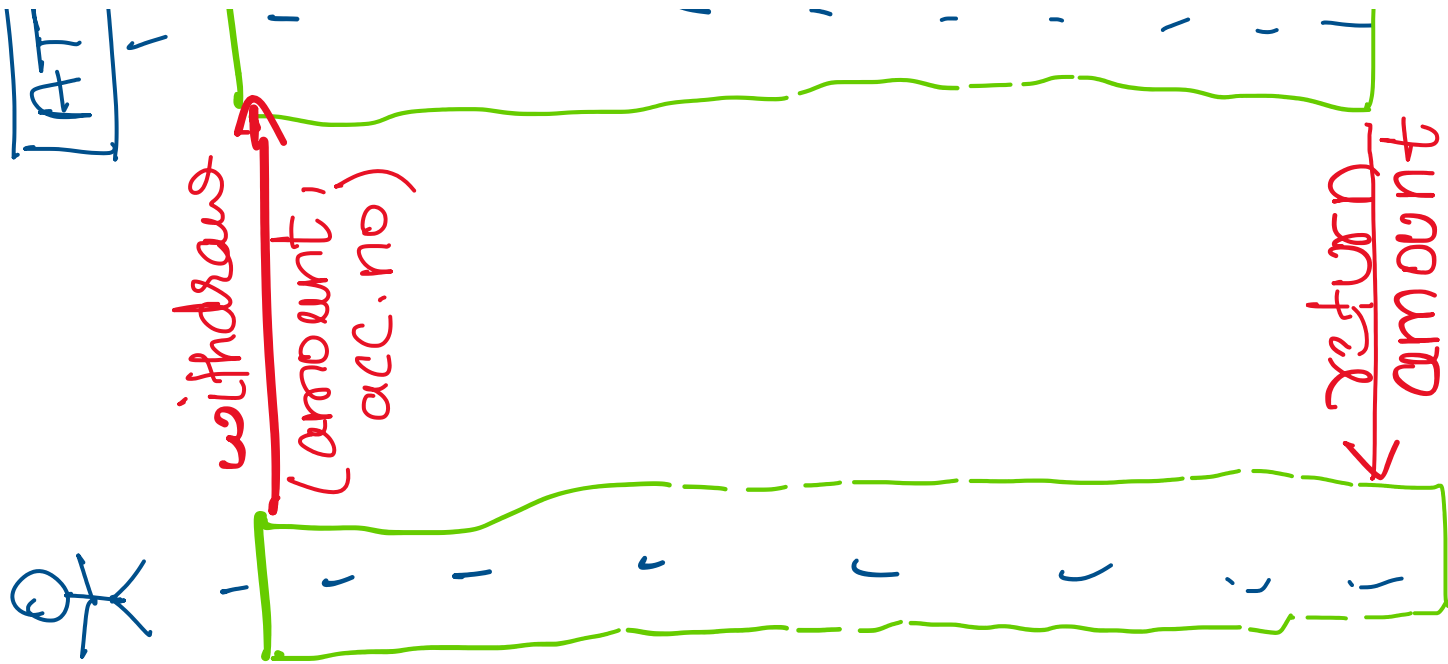
P.T.O.

Sequence diagram

Wednesday, 16 September 2026

11:51 PM





Some other Terms in Sequence Diagram

1. Alt — (if - else) block

↳ used when your entity returns false.

↳ Generally we discuss happy flow like in previous example

2. Option — (only if)

3. loop — for, while loop.